WEEK 3- ASSIGNMENT

Objective:

Use Subqueries, CTEs, and Window Functions to analyze sales data from the Superstore dataset.

Tasks:

Step 1: Setup Data

1. Import the Superstore dataset into a table named **superstore_raw**.
   - Downloaded the Superstore dataset (Sample - Superstore.csv).
   - Developed a Python script named **main.py**.
   - Used **Pandas** to read the CSV file.
   - Connected Python to MySQL database **csi3** using **MySQL Connector**.
   - Created the table **superstore_raw** dynamically.
   - Imported all records from the CSV file into MySQL.
   - Verified successful data import using row count validation.
   - Uploaded the Python script (**main.py**) to GitHub.

   Output:

```
Data imported successfully.
Database : csi3
Table    : superstore_raw
Rows     : 9994
```

2. Create Tables and Insert data into these tables using **SELECT DISTINCT**:
   a. Customers:

```
CREATE TABLE customers (
   customer_id VARCHAR(50) PRIMARY KEY,
   customer_name VARCHAR(100),
   segment VARCHAR(50)
);

-- Data Insert
INSERT INTO customers
SELECT DISTINCT
   customer_id,
   customer_name,
   segment
FROM superstore_raw;

-- View
select * from customers;
```

customer_id	customer_name	segment
AA-10315	Alex Avila	Consumer
AA-10375	Allen Armold	Consumer
AA-10480	Andrew Allen	Consumer
AA-10645	Anna Andreadi	Consumer
AB-10015	Aaron Bergman	Consumer
AB-10060	Adam Bellavance	Home Office
AB-10105	Adrian Barton	Consumer
AB-10150	Aimee Bixby	Consumer
AB-10165	Alan Barnes	Consumer
AB-10255	Alejandro Ballentine	Home Office

WEEK 3- ASSIGNMENT

b. Products:

```
CREATE TABLE products (
   product_id VARCHAR(50) PRIMARY KEY,
   product_name VARCHAR(255),
   category VARCHAR(100),
   sub_category VARCHAR(100)
);

--Data Insert
INSERT INTO products
SELECT
   product_id,
   MAX(product_name),
   MAX(category),
   MAX(sub_category)
FROM superstore_raw
GROUP BY product_id;

--View
select * from products;
```

product_id	product_name	category	sub_category
FUR-BO-10000112	Bush Birmingham Collection Bookcase, Dark Cherry	Furniture	Bookcases
FUR-BO-10000330	Sauder Camden County Barrister Bookcase, Pla...	Furniture	Bookcases
FUR-BO-10000362	Sauder Inglewood Library Bookcases	Furniture	Bookcases
FUR-BO-10000468	O'Sullivan 2-Shelf Heavy-Duty Bookcases	Furniture	Bookcases
FUR-BO-10000711	Hon Metal Bookcases, Gray	Furniture	Bookcases
FUR-BO-10000780	O'Sullivan Plantations 2-Door Library in Landver...	Furniture	Bookcases
FUR-BO-10001337	O'Sullivan Living Dimensions 2-Shelf Bookcases	Furniture	Bookcases
FUR-BO-10001519	O'Sullivan 3-Shelf Heavy-Duty Bookcases	Furniture	Bookcases
FUR-BO-10001567	Bush Westfield Collection Bookcases, Dark Cher...	Furniture	Bookcases
FUR-BO-10001601	Sauder Mission Library with Doors, Fruitwood Fi...	Furniture	Bookcases
FUR-BO-10001608	Hon Metal Bookcases, Black	Furniture	Bookcases
FUR-BO-10001619	O'Sullivan Cherrywood Estates Traditional Book...	Furniture	Bookcases
FUR-BO-10001798	Bush Somerset Collection Bookcase	Furniture	Bookcases
FUR-BO-10001811	Atlantic Metals Mobile 5-Shelf Bookcases, Custo...	Furniture	Bookcases
FUR-BO-10001918	Sauder Forest Hills Library with Doors, Woodlan...	Furniture	Bookcases

c. Orders:

```
CREATE TABLE orders (
   row_id INT PRIMARY KEY,
   order_id VARCHAR(50),
   order_date DATE,
   customer_id VARCHAR(50),
   product_id VARCHAR(50),
   sales DECIMAL(12,2),
   quantity INT,
   discount DECIMAL(5,2),
   profit DECIMAL(12,2)
);
```

WEEK 3- ASSIGNMENT

```
--Data Insert
INSERT INTO orders
SELECT
   CAST(row_id AS UNSIGNED),
   order_id,
   STR_TO_DATE(order_date,'%m/%d/%Y'),
   customer_id,
   product_id,
   REPLACE(sales, ',', '') + 0,
   CAST(quantity AS UNSIGNED),
   REPLACE(discount, ',', '') + 0,
   REPLACE(profit, ',', '') + 0
FROM superstore_raw;

--View
select * from orders;
```

row_id	order_id	order_date	customer_id	product_id	sales	quantity	discount	profit
1	CA-2016-152156	2016-11-08	CG-12520	FUR-BO-10001798	261.96	2	0.00	41.91
2	CA-2016-152156	2016-11-08	CG-12520	FUR-CH-10000454	731.94	3	0.00	219.58
3	CA-2016-138688	2016-06-12	DV-13045	OFF-LA-10000240	14.62	2	0.00	6.87
4	US-2015-108966	2015-10-11	SO-20335	FUR-TA-10000577	957.58	5	0.45	-383.03
5	US-2015-108966	2015-10-11	SO-20335	OFF-ST-10000760	22.37	2	0.20	2.52
6	CA-2014-115812	2014-06-09	BH-11710	FUR-FU-10001487	48.86	7	0.00	14.17
7	CA-2014-115812	2014-06-09	BH-11710	OFF-AR-10002833	7.28	4	0.00	1.97
8	CA-2014-115812	2014-06-09	BH-11710	TEC-PH-10002275	907.15	6	0.20	90.72
9	CA-2014-115812	2014-06-09	BH-11710	OFF-BI-10003910	18.50	3	0.20	5.78
10	CA-2014-115812	2014-06-09	BH-11710	OFF-AP-10002892	114.90	5	0.00	34.47
11	CA-2014-115812	2014-06-09	BH-11710	FUR-TA-10001539	1706.18	9	0.20	85.31
12	CA-2014-115812	2014-06-09	BH-11710	TEC-PH-10002033	911.42	4	0.20	68.36
13	CA-2017-114412	2017-04-15	AA-10480	OFF-PA-10002365	15.55	3	0.20	5.44
14	CA-2016-161389	2016-12-05	IM-15070	OFF-BI-10003656	407.98	3	0.20	132.59
15	US-2015-118983	2015-11-22	HP-14815	OFF-AP-10002311	68.81	5	0.80	-123.86

Step 2: Perform Required Queries

1. Orders with Sales Greater than Average Sales (Subquery):

```
SELECT *
FROM orders
WHERE sales >
(
   SELECT AVG(sales)
   FROM orders
);
```

Result:

row_id	order_id	order_date	customer_id	product_id	sales	quantity	discount	profit
1	CA-2016-152156	2016-11-08	CG-12520	FUR-BO-10001798	261.96	2	0.00	41.91
2	CA-2016-152156	2016-11-08	CG-12520	FUR-CH-10000454	731.94	3	0.00	219.58
4	US-2015-108966	2015-10-11	SO-20335	FUR-TA-10000577	957.58	5	0.45	-383.03
8	CA-2014-115812	2014-06-09	BH-11710	TEC-PH-10002275	907.15	6	0.20	90.72
11	CA-2014-115812	2014-06-09	BH-11710	FUR-TA-10001539	1706.18	9	0.20	85.31
12	CA-2014-115812	2014-06-09	BH-11710	TEC-PH-10002033	911.42	4	0.20	68.36
14	CA-2016-161389	2016-12-05	IM-15070	OFF-BI-10003656	407.98	3	0.20	132.59
17	CA-2014-105893	2014-11-11	PK-19075	OFF-ST-10004186	665.88	6	0.00	13.32
25	CA-2015-106320	2015-09-25	EB-13870	FUR-TA-10000577	1044.63	3	0.00	240.26
28	US-2015-150630	2015-09-17	TB-21520	FUR-BO-10004834	3083.43	7	0.50	-1665.05
36	CA-2016-117590	2016-12-08	GH-14485	TEC-PH-10004977	1097.54	7	0.20	123.47
39	CA-2015-117415	2015-12-27	SN-20710	FUR-BO-10002545	532.40	3	0.32	-46.98
41	CA-2015-117415	2015-12-27	SN-20710	TEC-PH-10000486	371.17	4	0.20	41.76
55	CA-2016-105816	2016-12-11	JM-15265	TEC-PH-10002447	1029.95	5	0.00	298.69
58	CA-2016-111682	2016-06-17	TB-21055	FUR-CH-10003968	319.41	5	0.10	7.10
68	CA-2014-106376	2014-12-05	BS-11590	OFF-AR-10002671	1113.02	8	0.20	111.30
73	US-2015-134026	2015-04-26	JE-15745	FUR-CH-10000513	831.94	8	0.20	-114.39

By: Pranjal Upadhyay

WEEK 3- ASSIGNMENT

2. Highest Sales Order for Each Customer (Subquery):

```
SELECT *
FROM orders o
WHERE sales =
(
   SELECT MAX(sales)
   FROM orders o2
   WHERE o.customer_id = o2.customer_id
);
```

Result:

row_id	order_id	order_date	customer_id	product_id	sales	quantity	discount	profit
2	CA-2016-152156	2016-11-08	CG-12520	FUR-CH-10000454	731.94	3	0.00	219.58
4	US-2015-108966	2015-10-11	SO-20335	FUR-TA-10000577	957.58	5	0.45	-383.03
11	CA-2014-115812	2014-06-09	BH-11710	FUR-TA-10001539	1706.18	9	0.20	85.31
25	CA-2015-106320	2015-09-25	EB-13870	FUR-TA-10000577	1044.63	3	0.00	240.26
28	US-2015-150630	2015-09-17	TB-21520	FUR-BO-10004834	3083.43	7	0.50	-1665.05
36	CA-2016-117590	2016-12-08	GH-14485	TEC-PH-10004977	1097.54	7	0.20	123.47
39	CA-2015-117415	2015-12-27	SN-20710	FUR-BO-10002545	532.40	3	0.32	-46.98
55	CA-2016-105816	2016-12-11	JM-15265	TEC-PH-10002447	1029.95	5	0.00	298.69
58	CA-2016-111682	2016-06-17	TB-21055	FUR-CH-10003968	319.41	5	0.10	7.10
68	CA-2014-106376	2014-12-05	BS-11590	OFF-AR-10002671	1113.[illegible]	[illegible]	0.20	111.30
110	CA-2015-129476	2015-10-15	PA-19060	TEC-AC-10000844	339.96	5	0.20	67.99

3. Total Sales for Each Customer (CTE):

```
WITH customer_sales AS
(
   SELECT
      customer_id,
      SUM(sales) AS total_sales
   FROM orders
   GROUP BY customer_id
)
SELECT *
FROM customer_sales;
```

Result:

customer_id	total_sales
CG-12520	1148.78
DV-13045	1119.48
SO-20335	2602.58
BH-11710	6255.34
AA-10480	1790.51
IM-15070	4930.49
HP-14815	1990.31
PK-19075	8646.93
AG-10270	2582.90
ZD-21925	1493.94

4. Customers with Above Average Sales (CTE + Subquery):

```
WITH customer_sales AS
(
   SELECT
      customer_id,
      SUM(sales) AS total_sales
   FROM orders
   GROUP BY customer_id
)

SELECT *
FROM customer_sales
WHERE total_sales >
(
   SELECT AVG(total_sales)
   FROM customer_sales
);
```

Result:

customer_id	total_sales
BH-11710	6255.34
IM-15070	4930.49
PK-19075	8646.93
TB-21520	4737.48
MA-17560	4299.16
SN-20710	3127.97
LC-16930	4492.94
RA-19885	3832.32
ES-14080	4657.93
KM-16720	4909.47
KD-16270	8282.36

5. Rank Customers Based on Total Sales (Window Function):

```
WITH customer_sales AS
(
   SELECT
      customer_id,
      SUM(sales) AS total_sales
   FROM orders
   GROUP BY customer_id
)
SELECT
   customer_id,
   total_sales,
   RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
FROM customer_sales;
```

Result:

customer_id	total_sales	sales_rank
SM-20320	25043.07	1
TC-20980	19052.22	2
RB-19360	15117.35	3
TA-21385	14595.62	4
AB-10105	14473.57	5
KL-16645	14175.23	6
SC-20095	14142.34	7
HL-15040	12873.30	8
SE-20110	12209.44	9
CC-12370	12129.08	10
TS-21370	11891.75	11

WEEK 3- ASSIGNMENT

6. Row Number for Each Order within Customer:

```
SELECT
   customer_id,
   order_id,
   sales,
   ROW_NUMBER() OVER
   (
      PARTITION BY customer_id
      ORDER BY order_date
   ) AS order_number
FROM orders;
```

Result:

customer_id	order_id	sales	order_number
AA-10315	CA-2014-128055	52.98	1
AA-10315	CA-2014-128055	673.57	2
AA-10315	CA-2014-138100	14.94	3
AA-10315	CA-2014-138100	14.56	4
AA-10315	CA-2015-121391	26.96	5
AA-10315	CA-2016-103982	3930.07	6
AA-10315	CA-2016-103982	431.98	7
AA-10315	CA-2016-103982	2.30	8
AA-10315	CA-2016-103982	41.72	9
AA-10315	CA-2017-147039	11.54	10
AA-10315	CA-2017-147039	362.94	11

7. Top 3 Customers Based on Total Sales:

```
WITH customer_sales AS
(
   SELECT
      customer_id,
      SUM(sales) AS total_sales
   FROM orders
   GROUP BY customer_id
)
SELECT *
FROM
(
   SELECT
      customer_id,
      total_sales,
      RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
   FROM customer_sales
) x
WHERE sales_rank <= 3;
```

Output:

customer_id	total_sales	sales_rank
SM-20320	25043.07	1
TC-20980	19052.22	2
RB-19360	15117.35	3

WEEK 3- ASSIGNMENT

Step 3: Final Combined Query:

- Customer Name + Total Sales + Rank:

```
WITH customer_sales AS
(
  SELECT
    customer_id,
    SUM(sales) AS total_sales
  FROM orders
  GROUP BY customer_id
)

SELECT
  c.customer_name,
  cs.total_sales,
  RANK() OVER (ORDER BY cs.total_sales DESC) AS customer_rank
FROM customer_sales cs
JOIN customers c
ON cs.customer_id = c.customer_id;
```

Result:

customer_name	total_sales	customer_rank
Sean Miller	25043.07	1
Tamara Chand	19052.22	2
Raymond Buch	15117.35	3
Tom Ashbrook	14595.62	4
Adrian Barton	14473.57	5
Ken Lonsdale	14175.23	6
Sanjit Chand	14142.34	7
Hunter Lopez	12873.30	8
Sanjit Engle	12209.44	9
Christopher Conant	12129.08	10
Todd Sumrall	11891.75	11

Mini Project: Customer Sales Insights:

1. **Top 5 Customers:**

```
WITH customer_sales AS
(
  SELECT
    c.customer_name,
    SUM(o.sales) AS total_sales
```

```
    FROM customers c
    JOIN orders o
    ON c.customer_id = o.customer_id
    GROUP BY c.customer_name
)

SELECT *
FROM customer_sales
ORDER BY total_sales DESC
LIMIT 5;
```

customer_name	total_sales
Sean Miller	25043.07
Tamara Chand	19052.22
Raymond Buch	15117.35
Tom Ashbrook	14595.62
Adrian Barton	14473.57

2. **Bottom 5 Customers:**

```
WITH customer_sales AS
(
  SELECT
    c.customer_name,
    SUM(o.sales) AS total_sales
  FROM customers c
  JOIN orders o
  ON c.customer_id = o.customer_id
  GROUP BY c.customer_name
)

SELECT *
FROM customer_sales
ORDER BY total_sales ASC
LIMIT 5;
```

Result:

customer_name	total_sales
Thais Sissman	4.84
Lela Donovan	5.30
Carl Jackson	16.52
Mitch Gastineau	16.74
Roy Skaria	22.33

WEEK 3- ASSIGNMENT

3. Customers Who Made Only One Order

```
SELECT
   c.customer_name,
   COUNT(DISTINCT o.order_id) AS total_orders
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name
HAVING COUNT(DISTINCT o.order_id) = 1;
```

Result:

customer_name	total_orders
Anemone Ratner	1
Anthony O'Donnell	1
Carl Jackson	1
Jenna Caffey	1
Jocasta Rupert	1
Lela Donovan	1
Mitch Gastineau	1
Patricia Hirasaki	1
Ricardo Emerson	1
Roland Murray	1

4. Customers with Above Average Sales:

```
WITH customer_sales AS
(
   SELECT
      c.customer_name,
      SUM(o.sales) AS total_sales
   FROM customers c
   JOIN orders o
   ON c.customer_id = o.customer_id
   GROUP BY c.customer_name
)

SELECT *
FROM customer_sales
WHERE total_sales >
(
   SELECT AVG(total_sales)
   FROM customer_sales
);
```

Result:

customer_name	total_sales
Brosina Hoffman	6255.34
Irene Maddox	4930.49
Pete Kriz	8646.93
Tracy Blumstein	4737.48
Matt Abelman	4299.16
Steve Nguyen	3127.97
Linda Cazamias	4492.94
Ruben Ausman	3832.32
Erin Smith	4657.93
Kunst Miller	4909.47
Karen Daniels	8282.36

5. Highest Order Value Per Customer:

```
SELECT
   c.customer_name,
   MAX(o.sales) AS highest_order_value
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name
ORDER BY highest_order_value DESC;
```

Result:

customer_name	highest_order_value
Sean Miller	22638.48
Tamara Chand	17499.95
Raymond Buch	13999.96
Tom Ashbrook	11199.97
Hunter Lopez	10499.97
Adrian Barton	9892.74
Sanjit Chand	9449.95
Bill Shonely	9099.93
Sanjit Engle	8749.95
Christopher Conant	8399.98
Ken Lonsdale	8187.65

Brief Insights:

1. A small group of customers contributes a large percentage of total sales.
2. Several customers place only one order, indicating opportunities for retention campaigns.
3. Above-average customers are high-value customers and should be targeted with loyalty programs.
4. Ranking customers helps identify VIP customers and revenue drivers.
5. Highest-order-value analysis helps understand large purchase behavior and premium customer segments.